
LIVRET DE BORD

PROJET ANNUEL 3IABD -2024/2025

Traduction Intelligente de la langue des signes

Membre du groupe :

Khaoula CHETIOUI

Asma MOUGNI

Thinhinane AKRICHE

Classe: 3IABD2

Encadrant : Nicolas VIDAL

1. Présentation du projet :

SignAI - Traduction intelligente de la langue des signes

Le projet SignAI vise à développer un système de reconnaissance de la langue des signes basé sur des modèles de machine learning implémentés en Rust. L'objectif principal est de classer des images représentant différentes lettres ou chiffres en langue de signes. Cette initiative répond au besoin croissant d'intégrer la langue des signes dans les technologies numériques pour améliorer l'accessibilité des personnes sourdes et malentendantes.

2. Objectifs principaux

2.1 Développer une bibliothèque de Machine Learning en Rust

Une bibliothèque Rust optimisée assurera l'exécution des algorithmes de reconnaissances des signes avec performance et fiabilité

Modèles à implémenter :

- Modèle Linéaire : pour les classifications de base
- Perceptron Multi-Couches (PMC) : pour les classifications complexes
- Réseau à base Radiale (RBF) : pour une classification avec des fonctions de base radiale
- Support Vector Machine : pour des classifications à haute dimensionnalité

2.2 Créer une application Web interactive

L'application web sert de plateforme utilisateur, permettant une interaction avec le système de traduction automatique de la langue des signes. Elle est développée avec un **frontend en Rust**, qui fournit une interface réactive et performante, et un **backend en FastAPI (Python)**, choisi pour sa légèreté et sa capacité à s'interfacer facilement avec des APIs externes

Architecture technique

Backend FastAPI : gère l'authentification, les appels à l'API de prédiction et le stockage des requêtes utilisateur. Il expose des endpoints REST accessibles par le frontend.

Frontend Rust : constitue l'interface utilisateur. L'utilisateur peut y téléverser une image pour tester la reconnaissance de gestes. L'image est ensuite envoyée au backend via une requête HTTP.

Connexion avec l'API de prédiction : le backend transmet les images reçues à une API de prédiction (en Rust ou autre), via des appels REST.

Traitement des images :

- L'utilisateur téléverse une image via le frontend.
- Le backend FastAPI reçoit l'image et l'envoie à l'API de prédiction.
- L'API renvoie le résultat (texte ou interprétation du geste).
- Le frontend affiche dynamiquement la traduction à l'utilisateur.

2.3 Créer un dataset original

Dans le cadre de ce projet de reconnaissance de la langue des signes, un dataset personnalisé a été conçu afin de répondre aux besoins spécifiques d'un modèle de classification supervisée. Ce dataset repose sur un ensemble d'images représentant trois gestes manuels issus de la Langue des Signes Française, correspondant aux lettres A, B et C

L'objectif de cette phase est de construire un jeu de données simple, structuré et exploitable pour entraîner un modèle de machine learning dans un contexte contrôlé. Plus précisément, ce dataset vise à :

- Couvrir trois classes représentées par les lettres A, B et C, dans le but de tester le fonctionnement initial du système de reconnaissance.
- Structurer les données dans un fichier CSV, où :
 - Chaque ligne correspond à une image,
 - Chaque colonne contient une valeur de pixel (format RGB),
 - Une dernière colonne indique le label associé à l'image (ex : "A", "B", ou "C").
- Utiliser ce dataset pour entraîner, valider et tester un premier modèle de classification

Méthodologie de collecte

Les images sont collectées selon deux méthodes principales :

- Photos personnelles : des images ont été prises manuellement en reproduisant les gestes A, B et C avec les mains, dans différentes conditions (lumière, arrière-plan, position).

- Recherche sur internet : d'autres images ont été récupérées via des moteurs de recherche (ex. Google Images) et sites spécialisés, en sélectionnant uniquement des représentations claires et correctes des gestes de la langue des signes

Structure du dataset

Le dataset est organisé de manière **hiérarchique selon les classes** (A, B, C), et divisé en deux sous-ensembles distincts :

- **Entraînement (train)** : 1200 images par classe, soit 3600 images au total. Cette partie représente environ **80% du dataset** et est utilisée pour apprendre les caractéristiques visuelles des gestes.
- **Test (test)** : 300 images par classe, soit 900 images au total. Cette partie constitue environ **20% du dataset** et contient uniquement des images **jamais vues** pendant l'entraînement. Elle permet d'évaluer la **capacité de généralisation** du modèle.

Dataset/

```

├── train/
│   ├── A/
│   │   ├── A_001.jpg
│   │   ├── A_002.jpg
│   │   └── ...
│   ├── B/
│   │   ├── B_001.jpg
│   │   └── ...
├── test/
│   ├── A/
│   ├── B/
│   └── ..

```

Partie 1 : Modèle implémentés

Modèle linéaire :

Les modèles linéaires constituent une base incontournable en apprentissage supervisé. Plusieurs variantes ont été implémentées pour répondre à différents besoins : régression, classification binaire, apprentissage itératif ou analytique.

Partie 01 : Régression et classification binaire

Cette section présente les différentes approches que nous avons expérimentées pour l'apprentissage supervisé dans le cadre de la régression et de la classification binaire. Nous avons implémenté quatre variantes de modèles linéaires, chacun correspondant à une stratégie d'entraînement différente :

Objectif:

Explorer et comparer plusieurs méthodes d'apprentissage pour les modèles linéaires afin de comprendre :

- leurs comportements numériques (stabilité, convergence),
- leurs limitations (cas de matrices non inversibles),
- leur capacité à gérer ou non des fonctions d'activation non linéaires.

1. Pseudo-Inverse (Méthode Analytique via SVD) : Fichier `linear_pseudo_inverse.rs`

Description

La pseudo-inverse permet de résoudre analytiquement le problème de régression linéaire :

$$W = (X^T * X)^{-1} * X^T * Y$$

Mais ici, on utilise une version plus robuste basée sur la **décomposition SVD** avec nalgebra, qui permet une inversion généralisée même lorsque $X^T X$ est mal conditionnée.

Avantages

- Méthode directe (pas d'itération).
- Robuste avec SVD (comparée aux moindres carrés simples).

Limite importante :

On ne peut pas appliquer de fonction d'activation lors de l'apprentissage : L'activation est appliquée seulement à la prédiction, car l'entraînement est une solution fermée, analytique et linéaire.

Avec l'exemple Tricky 3D (donnée par le prof) la méthode de moindre carrée simple sans la gestion SVD échoue mais avec la pseudo inverse donne une précision de 100%.

```
Linear Tricky 3D :  
  
Linear Model : OK (la moindre carrée échoue car la matrice est non inversible)  
  
>   
x = np.array([  
    [1, 1],  
    [2, 2],  
    [3, 3]  
)  
  
#  $X^T X = \begin{bmatrix} 14 & 14 \end{bmatrix}$ ,  
#  $\begin{bmatrix} 14 & 14 \end{bmatrix}$ ,  
#  $\det(X^T X) = 0 \rightarrow$  non inversible (la moindre carrée échoue)  
y = np.array([  
    1,  
    2,  
    3  
)
```

Étapes de la méthode :

Ajout du biais : On étend la matrice X pour intégrer le biais b, en ajoutant une colonne de 1 en début de chaque ligne .

Formulation matricielle du problème : On cherche à résoudre : $X_{\text{extend}} \cdot w_{\text{extend}} = y$:

Résolution avec SVD (décomposition en valeurs singulières) :

$$X_{\text{extend}} = U \Sigma V^T \text{ ou :}$$

$$U \in \mathbb{R}(n \times n)$$

$$U \in \mathbb{R}(n \times (d+1))$$

$$U \in \mathbb{R}((d+1) \times (d+1)) \text{ (Dans le code elle est calculé directement par la fonction .svd fournit par nalgebra)}$$

$$\text{Calcul final des poids : } W_{\text{extend}} = X_{\text{extend}} \cdot y$$

2. Moindres Carrés Ordinaires : Fichier : **linear_least_squares.rs**

Description

On applique la formule classique des moindres carrés :

$$W = (X^T * X)^{-1} * X^T * Y$$

Étapes de la méthode :

- 1 - Extension de la matrice avec le biais : Ajout d'une colonne de 1 pour inclure le biais.
- 2 - Calcul de la transposée : X^T .
- 3 - Produit matriciel : $X^T X$.
- 4 - Inversion : $(X^T X)^{-1}$ (Cette étape peut échouer si la matrice est non inversible ou mal conditionnée)
- 5 - Produit final : $w = (X^T X)^{-1} X^T Y$ (on obtient tous les poids, dont le biais $b = w_0$)

Particularité

Ici, on code manuellement les opérations matricielles :

- transposition,
- produit matriciel,
- inversion.

Cela permet de mieux comprendre la démarche mathématique.

Limite

Cette méthode échoue si la matrice $X^T X$ n'est pas inversible (ce qui arrive quand les colonnes de X sont linéairement dépendantes).

→ D'où l'intérêt d'utiliser la pseudo-inverse SVD pour les cas dégénérés.

Preuve avec l'exemple Tricky 3D :

```
plt.tight_layout()
ax.legend()
plt.tight_layout()
plt.show()
```

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](#) for more info.

View Jupyter [log](#) for further details.

Non Linear Simple 3D :

3- Descente de Gradient : Fichier : `linear_gradient_descent.rs`

Description

La descente de gradient est une méthode itérative qui permet :

- d'entraîner les poids progressivement,
- de prendre en compte des fonctions d'activation non linéaires.
- de travailler même lorsque l'inversion est impossible.

Fonctionnement

À chaque itération :

1. Calcul de la sortie brute (logits) : $z = w \cdot x + b$

(Cela correspond à un produit scalaire entre les poids et les entrées, auquel on ajoute le biais).

2. Application de la fonction d'activation : $y^{\wedge} = f(w \cdot x + b)$
3. On mesure la différence entre la prédiction et la vraie valeur cible : $e = y^{\wedge} - y$
4. Calcul du gradient (avec dérivée de l'activation) : $g = e \cdot f'(y^{\wedge})$
5. Mise à jour des poids :

$W_i = W_i - \text{learning_rate} * g * X_i$ (pour chaque poids)

$b = b - \text{learning_rate} * g$

Avantages

Compatible avec activation (sigmoid, tanh, relu)

S'adapte à toute fonction de coût

Robuste aux matrices non inversibles

Inconvénients

- Plus lent que les méthodes analytiques (car itératif)
- Sensible aux hyperparamètres (taux d'apprentissage, nombre d'époques)

4 - Méthode de Rosenblatt (Perceptron) : Fichier : `linear_rosenblat_method.rs`

Principe :

Il repose sur un principe simple : si un point est mal classé, on ajuste les poids dans la direction de l'erreur.

Objectif du modèle :

On cherche un vecteur de poids $w \in \mathbb{R}$ et un biais b qui permettent de classer des exemples dans deux classes -1 et $+1$ à l'aide d'une fonction de décision linéaire :

$$z = w \cdot x + b \quad \Rightarrow \quad y^{\wedge} = \{+1 \text{ si } z \geq 0 \text{ sinon : } -1 \text{ si } z < 0\}$$

Étapes de l'algorithme

1. Calcul du score linéaire (logit) :

$$z_i = w \cdot x_i + b$$

2. Prédiction via la fonction de seuil :

$$y^{\wedge} = \{+1 \text{ si } z \geq 0 \text{ sinon : } -1 \text{ si } z < 0\}$$

3. Erreur de classification :

$$e_i = y_i - y^{\wedge}_i$$

4. Mise à jour des poids (si l'exemple est mal classé) :

$$W_i = W_i - \text{learning_rate} * g * X_i \quad (\text{pour chaque poids})$$

$$b = b - \text{learning_rate} * g$$

Limitations

- Convergence uniquement si les données sont **linéairement séparables**
- Pas de fonction d'activation différentiable.
- Ne fonctionne pas bien dans le cas non-linéaire

Partie 02 : Classification multiclass

Modèle OneVsAllTanhMSEModel : tanh + MSE

Le modèle OneVsAllTanhMSEModel implémente une stratégie de classification multiclass basée sur le principe **One-vs-All**, où chaque classe est traitée indépendamment par un classifieur binaire. Pour chaque classe, une sortie est calculée en appliquant une combinaison linéaire des entrées (produit scalaire entre les poids et les features, plus un biais), puis transformée par la **fonction d'activation tanh**. Cette fonction permet d'obtenir une sortie continue entre -1 et 1, utile pour approximer une prédiction binaire même si les cibles sont {0, 1}.

L'apprentissage est supervisé par la fonction de perte MSE (Mean Squared Error). À chaque itération, les sorties du modèle sont comparées à un vecteur cible one-hot encodant la vraie classe, et l'erreur est rétropropagée. La dérivée de la loss par rapport à chaque sortie est combinée à la dérivée de tanh ($1 - \tanh^2(z)$) pour ajuster les poids et les biais par descente de gradient.

Au cours de l'entraînement, le modèle enregistre à chaque époque la perte moyenne (loss) et l'exactitude (accuracy) sur le jeu d'entraînement, et éventuellement sur le jeu de test si celui-ci est fourni. Cela permet de suivre l'évolution de l'apprentissage, de détecter un surapprentissage, et d'évaluer la performance du modèle.

La prédiction finale est obtenue en appliquant tanh à chaque score de classe, puis en choisissant la classe ayant la sortie maximale. Le modèle dispose aussi d'une fonction

predict_scores qui retourne les scores bruts pour toutes les classes, permettant une analyse plus fine.

Cette architecture simple et efficace permet d'entraîner rapidement un classifieur multiclasse entièrement en Rust, en combinant des idées classiques du machine learning (descente de gradient, One-vs-All, tanh, MSE) dans un cadre léger et personnalisable.

Architecture visuel de l'entraînement :

1. Initialisation : Poids et biais sont initialisés aléatoirement dans un intervalle petit (par exemple $[-0.01, 0.01]$) pour briser la symétrie.

2. Forward Pass : Pour chaque exemple x_i de dimension d et sa classe $y_i \in \{0, \dots, K-1\}$:

a. Calcul de la sortie brute (logits) : $z_i = w_i \cdot x_i + b_i = \sum_j w_{ij} * x_{ij} + b_i$

b. Application de la fonction d'activation tanh : $\hat{y}_i = \tanh(z_i)$

3. Création de la cible One-Hot : le vecteur cible t est défini comme : $t_k = 1$ si $k == y_i$, sinon 0

4. Calcul de la perte MSE : On utilise une version par sortie : $L = \frac{1}{2} * \sum_i (\hat{y}_i - t_i)^2$

5. Backpropagation / Mise à jour des paramètres :

5 - a Dérivée de la loss par rapport à la sortie : $\partial L / \partial \hat{y}_i = \hat{y}_i - t_i$

5 - b Dérivée de tanh : $\partial \hat{y}_i / \partial z_i = 1 - \tanh^2(z_i) = 1 - \hat{y}_i^2$

5 - c Gradient total : $\partial L / \partial z_i = (\hat{y}_i - t_i) * (1 - \hat{y}_i^2)$

5 - d Mise à jour des poids : $w_{ij} \leftarrow w_{ij} - \eta * \partial L / \partial z_i * x_{ij}$

5 - e Mise à jour des biais : $b_i \leftarrow b_i - \eta * \partial L / \partial z_i$

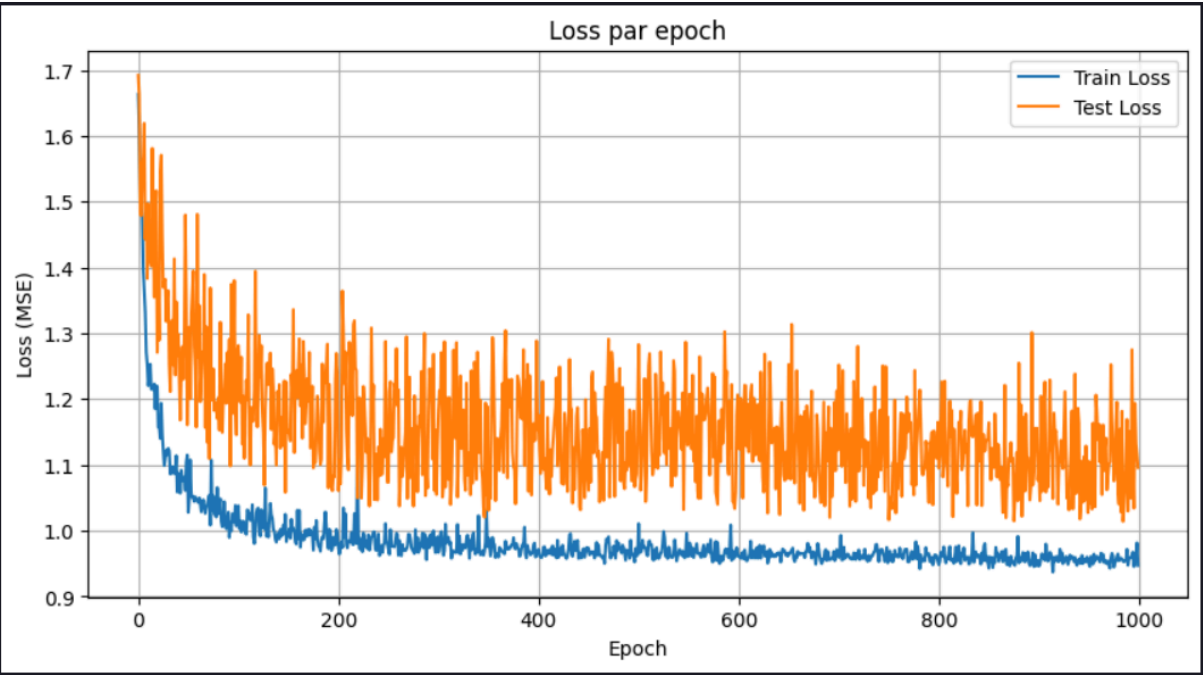
Test sur le dataset du PA :

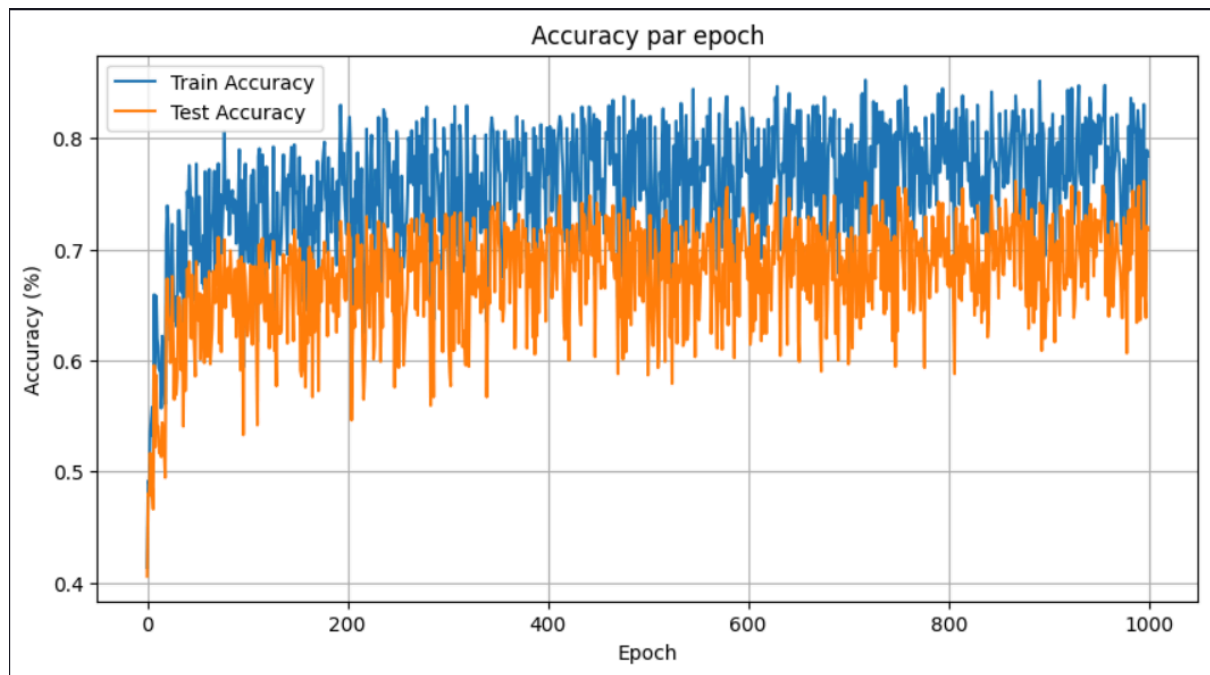
Tableau de résultats de tests avec différents paramètres sur le dataset :

Learning Rate	0.01	0.01	0.01	0.001
Epochs	1000	1000	2000	1000
Nombres d'images de tests par classes	10	310	310	310
Accuracy	0.29125063142066121	0.5912087912087912	0.60219780219780	0.454945054945054

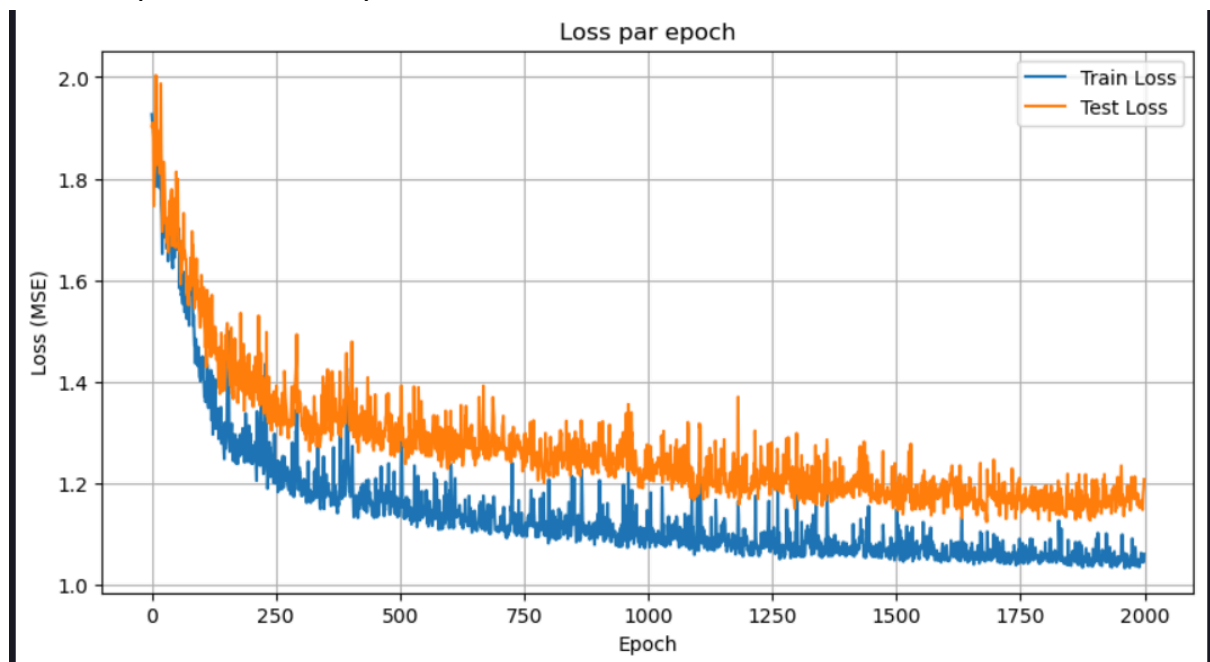
Courbes :

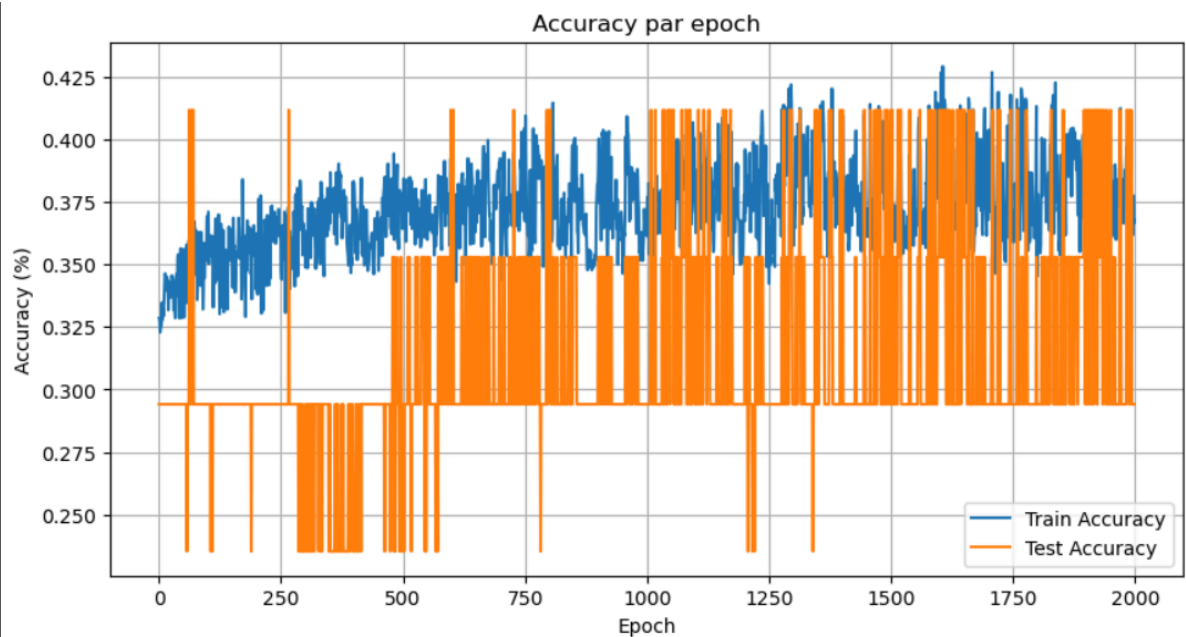
Cas 02 : (0.01 / 1000 / 310)



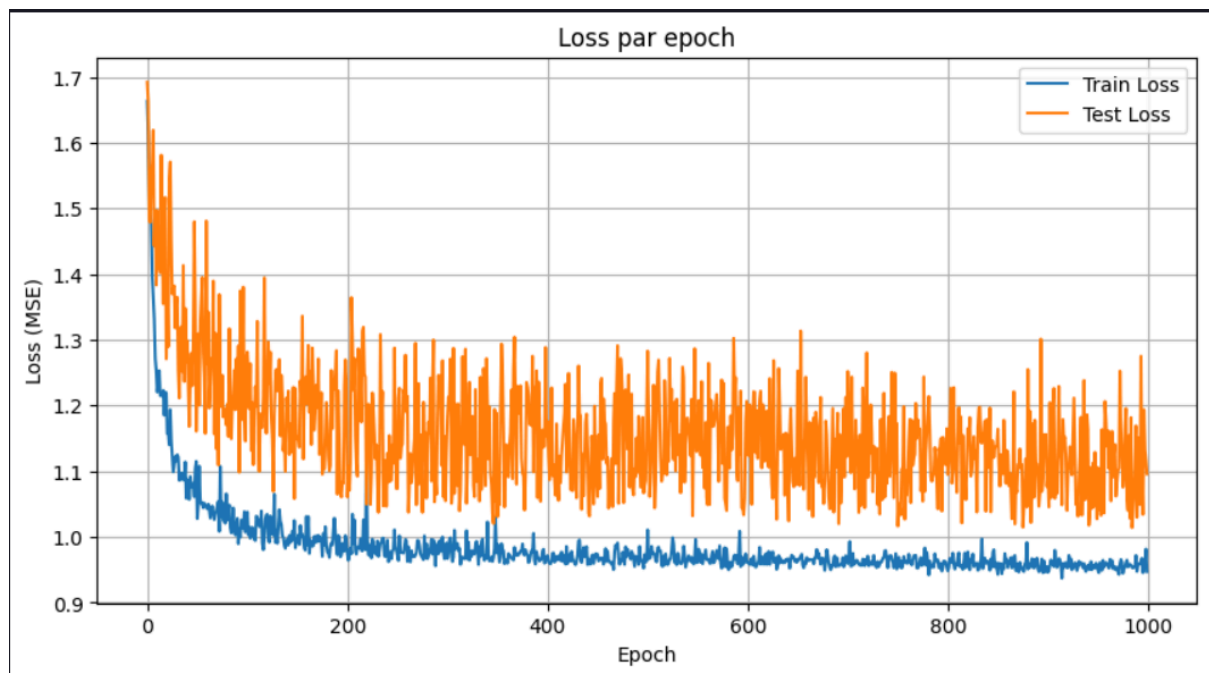


Cas 03 : (0.01 / 2000 / 310)





Cas 04 : (0.001 / 1000 / 310)



Analyse :

- Peu d'images de test (10 / classe) → accuracy = 0.29 : Échantillon trop petit ⇒ forte variance : ce score n'est pas représentatif.
- Doubler les époques n'apporte qu'≈ +1 pt → le modèle a quasiment convergé vers ~0.60.
- LR trop petite → sous-apprentissage (pas le temps de converger en 1000 époques).
-> avec ce LR, il faudrait beaucoup plus d'époques ou un scheduler.

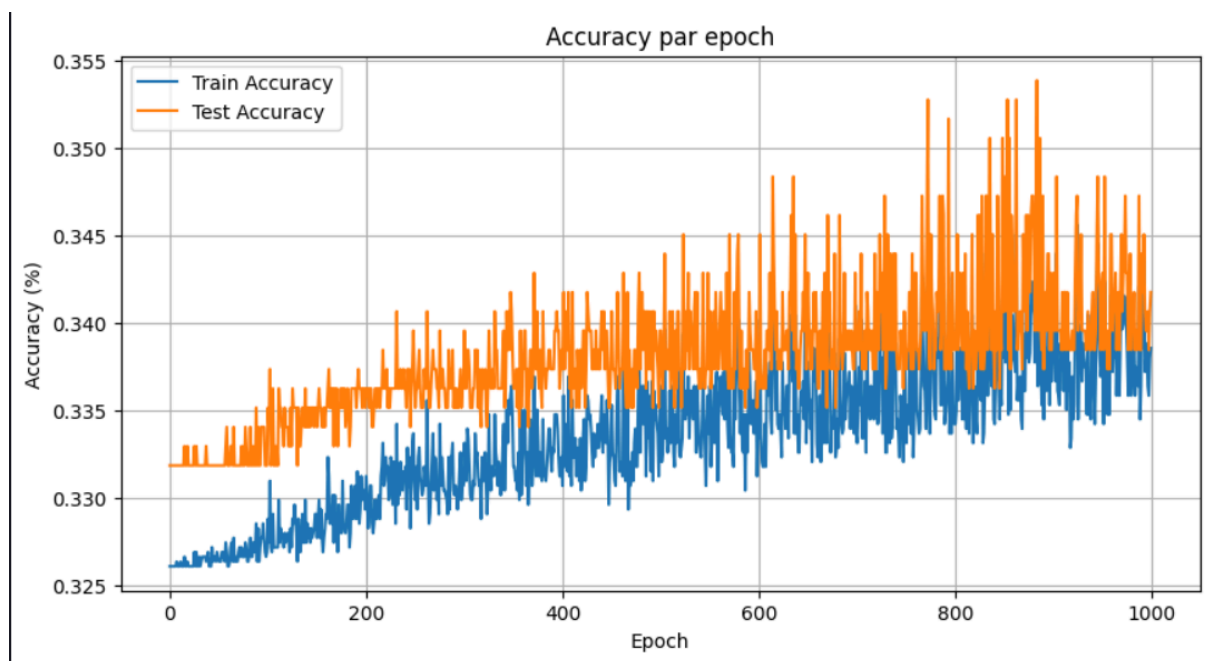
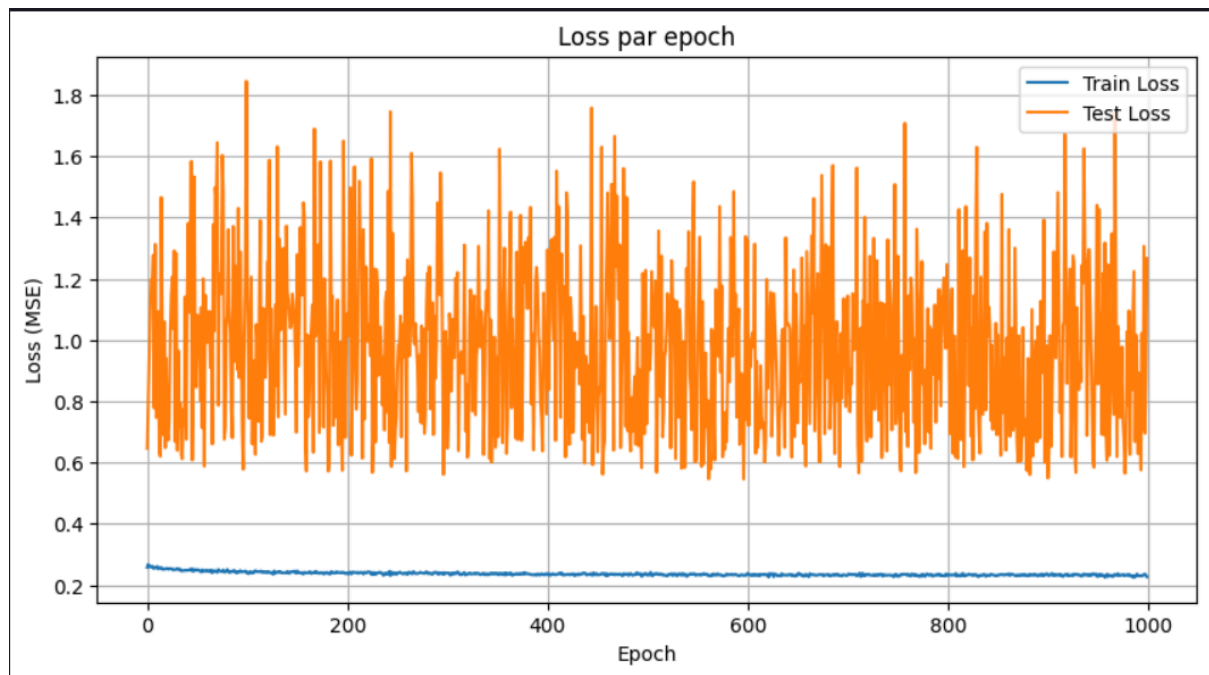
Problème rencontrés :

Lors des premiers tests, le modèle `OneVsAllTanhMSEModel` présentait un comportement anormal. Bien que les poids soient initialisés aléatoirement dans un intervalle relativement large $[-1.0, 1.0]$, l'apprentissage semblait se figer très rapidement : les poids convergeaient dès les premières dizaines d'époques vers des valeurs proches de zéro, et les prédictions restaient bloquées sur une seule classe. L'accuracy sur le jeu de test restait ainsi très faible (autour de 29%), indiquant que le modèle n'apprenait pas correctement.

On a dû ajouter une petite fonction au modèle rust pour afficher la mise à jour des poids pour savoir si ils s'améliorent ou pas :

```
Epoch 1/1000 — Loss moyenne: 0.0092
Poids[0][0]: -0.00106
Epoch 101/1000 — Loss moyenne: 0.0126
Poids[0][0]: -0.00088
Epoch 201/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 301/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 401/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 501/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 601/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 701/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 801/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Epoch 901/1000 — Loss moyenne: 0.0125
Poids[0][0]: -0.00089
Entraînement terminé !
✅ Accuracy sur données de test : 0.29411764705882354
```

Courbes de sortie :



Sur la courbe d'accuracy (train/test), on observe une croissance très lente, avec une accuracy plafonnée autour de 33%, aussi bien sur le jeu d'entraînement que sur le jeu de test. Ce plateau suggère que le modèle a appris à prédire systématiquement la même classe, sans capacité de généralisation. Cela est confirmé par l'absence d'évolution marquée entre les epochs, ainsi que par une grande proximité entre les courbes de test et de train.

Pour améliorer cela, une normalisation des données d'entrée a été introduite via `StandardScaler`, afin de centrer et réduire les features.

Afin d'améliorer les performances du modèle `OneVsAllTanhMSEModel`, nous avons étudié l'impact de la fonction d'activation et de la fonction de perte utilisées. Le modèle initial reposait sur l'utilisation de la fonction `tanh` combinée à une perte MSE, ce qui s'est révélé sous-optimal pour une tâche de classification multiclasse. Pour obtenir de meilleurs résultats, nous avons opté pour une approche plus adaptée : l'utilisation de la fonction `softmax` en sortie, couplée à la perte cross-entropy. Cette combinaison permet d'interpréter les sorties comme des probabilités normalisées et renforce la distinction entre les classes, ce qui conduit généralement à une convergence plus stable et à une amélioration significative de l'accuracy.

Modèle `SoftmaxModel` : `softmax` + `cross_entropy`

Le modèle `SoftmaxModel` est conçu pour résoudre des problèmes de classification multiclasse, où chaque entrée appartient à une seule classe parmi K possibles. Il repose sur une sortie `softmax`, interprétée comme une distribution de probabilités, et une fonction de perte cross-entropy, particulièrement adaptée à ce type de tâche.

Étapes de l'entraînement (méthode `fit`) :

1 - Calcul des scores (logits) pour chaque classe :

$$Z_k = \text{Somme} (W_{k,j} \cdot X_i + b_k)$$

2 - Application de la fonction `softmax` :

Cette opération transforme les scores bruts en une distribution de probabilité sur les classes.

3 - Calcul de la perte cross-entropy :

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Formule

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax
 $\vec{z}$ = vecteur d'entrée
 e^{z_i} = fonction exponentielle normalisée du vecteur d'entrée
 K = nombre de classes dans la classification en classes multiples
 e^{z_j} = fonction exponentielle normalisée du vecteur de sortie
 e^{z_j} = fonction exponentielle normalisée du vecteur de sortie

Résultats de test sur le dataset :

Learning Rate	0.01	0.001	0.1
Epochs	1000	600	600
Nombres d'images de tests par classes	310	310	310
Accuracy	0.743956043956044	0.7516483516483516	0.8351648351648352

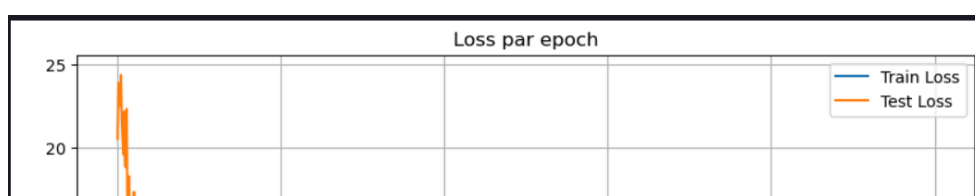
`learning_rate = 0.1` favorise une meilleure convergence rapide en seulement 600 epochs.

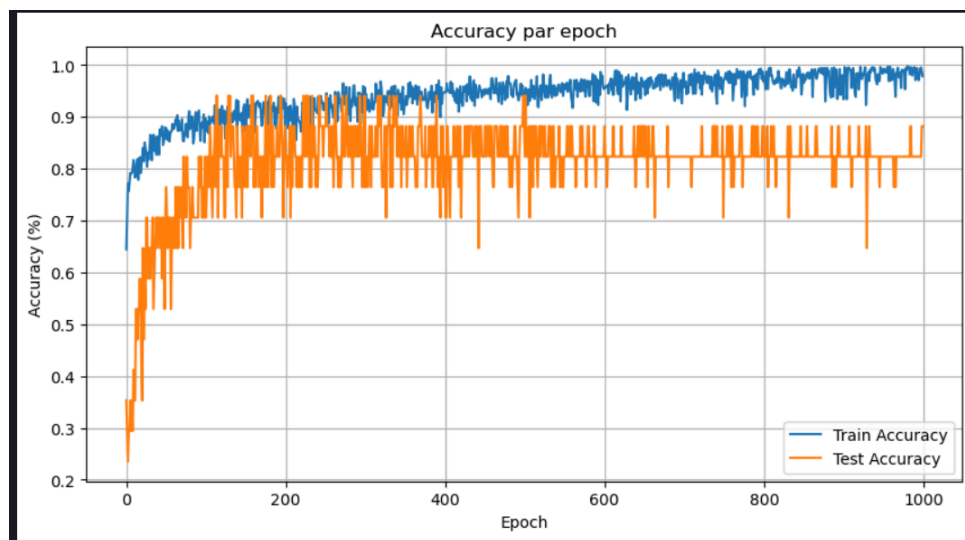
Analyse

1. Performance significativement meilleure avec softmax :

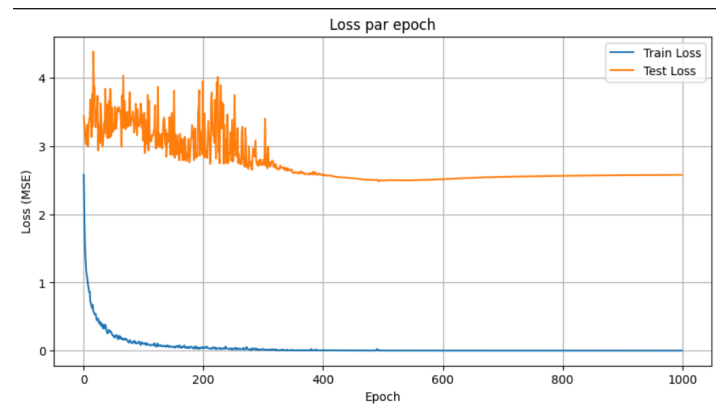
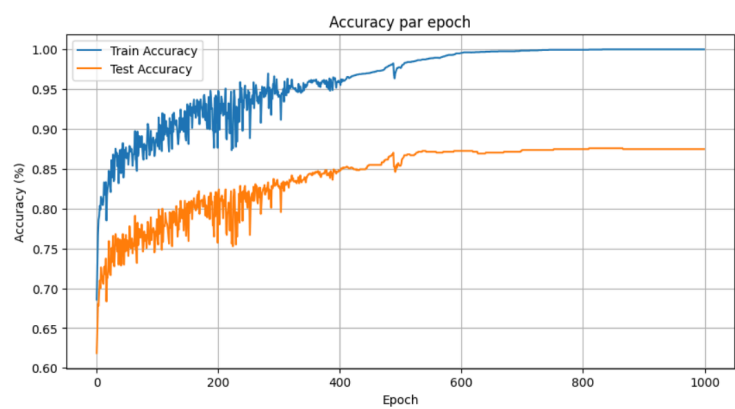
- Le passage du modèle tanh + MSE vers softmax + cross-entropy permet un gain net de plus de 20 points d'accuracy, à nombre d'images et epochs équivalents.
- Cela montre que tanh combiné à la MSE n'est pas adapté à une tâche de classification multiclasse native.

Cas 01 : (0.01, 1000, 310)





cas 02 : (0.1, 600, 310)



Le graphique montre que la perte sur le jeu d'entraînement (Train Loss) diminue rapidement et atteint quasiment zéro après 300 epochs. Cela signifie que le modèle parvient à mémoriser parfaitement les exemples d'entraînement, ce qui est typique d'un réseau bien entraîné, mais aussi potentiellement trop spécifique.

SVM (Support Vector Machine) avec noyau RBF

Un SVM c'est un algorithme de machine learning supervisé utilisé en général pour des tâches de classifications (binaire et multiclasse dans notre cas).

En principe le SVM cherche à séparer les classes en définissant des hyperplans dans le dataset.

Cet hyperplan est choisi de manière à pouvoir maximiser la marge, c'est-à-dire la distance qu'il peut y avoir entre les points les plus proches de chaque classe. Et donc en maximisant cette marge, le SVM devient plus robuste aux erreurs et va permettre de mieux généraliser les données.

Noyau RBF ?

Souvent, les données ne sont pas linéairement séparables et c'est donc pour cela que le noyau RBF intervient.

Le noyau va permettre de projeter les données dans un espace avec des dimensions plus grande et sans faire de calcul pour la projection, c'est possible grâce au kernel Trick.

5 - Partie SVM Multiclasse:

Pour l'instant, notre SVM ne sait gérer que deux classes à la fois, donc il fonctionne juste pour des problèmes où il n'y a que deux catégories possibles. Mais pour notre projet sur la reconnaissance de l'alphabet en langue des signes française, il nous faut pouvoir distinguer plein de lettres différentes, donc faire du multi-classes.

Pour ça, il existe deux techniques principales : One-vs-All (OvA) et One-vs-One (OvO).

- One-vs-All (OvA), c'est genre "un contre tous". Pour chaque classe, on entraîne un SVM qui essaie de séparer cette classe du reste. Quand on veut classer une nouvelle image, on fait passer l'exemple dans tous les SVM, et celui qui est le plus "sûr" de sa prédiction donne la classe finale.
- One-vs-One (OvO), c'est "un contre un". Là, on entraîne un SVM pour chaque paire de classes possible. Par exemple, pour 4 classes, on aurait 6 SVM (A vs B, A vs C, etc.). Quand on a une nouvelle image à classer, chaque SVM vote pour une des deux classes qu'il connaît, et à la fin, la classe qui a le plus de votes gagne.

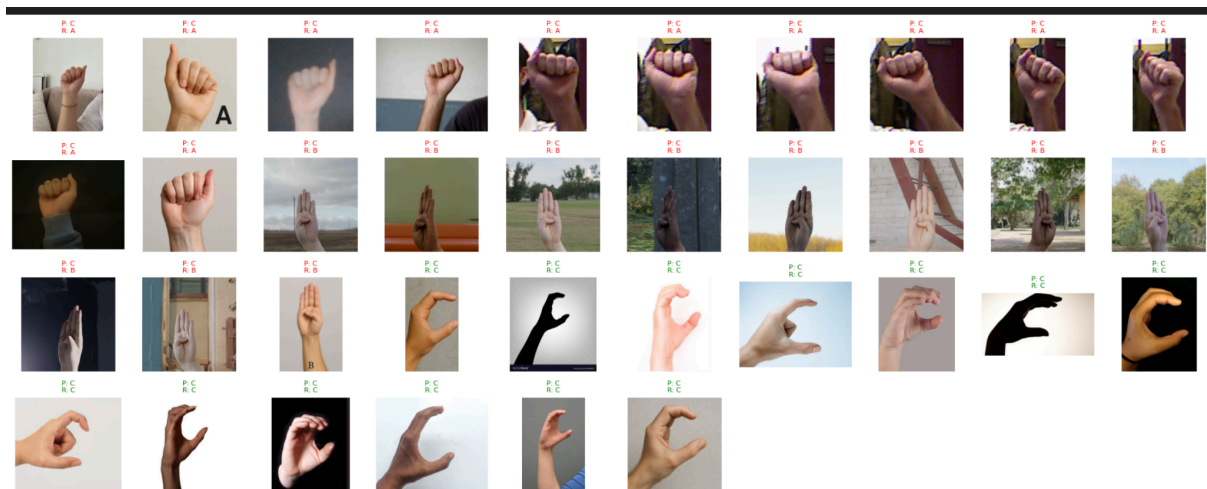
Nous avons choisi One-vs-All parce que c'est beaucoup plus simple à gérer. Pour chaque lettre, on a un SVM qui dit "est-ce que c'est cette lettre ou pas ?". Quand on veut reconnaître une nouvelle image, on la passe dans tous les SVM et celui qui est le plus confiant gagne.

Ça demande moins de ressources à entraîner et à stocker (seulement 26 modèles au lieu de 325), donc c'est plus pratique, surtout si on veut que le système soit léger et facile à maintenir.

Après avoir terminé de développer le svm multiclasse en utilisant la méthode One-vs-All, et nous avons déclaré les fonctions dans le fichier lib.rs. Nous effectuons les cas de test donné durant les séances de projet annuel pour s'assurer que l'algorithme fonctionne bien, après cela on effectue les premiers test sur le dataset et on remarque que le temps d'exécution est extrêmement long avec environ 8h d'attente donc nous avons commencé à réfléchir à ce qui pourrait poser un tel problème (le volume du dataset?, le cas de tests en python contiendrait peut être une boucle qui tourne dans le vide?, voir même un problème de composants dans nos ordinateurs.)

Nous avons donc décidé de réduire la taille du dataset a 3 classes et dans ces 3 classes 300 images. Et nous avons remarqué que les tests étaient beaucoup plus rapides dans leur exécution (environs 3 minutes), mais les résultats sur le cas de test qui porte sur notre

dataset ne prédisait que la lettre C partout ce qui nous laisse donc avec une accuracy d'un tiers seulement.



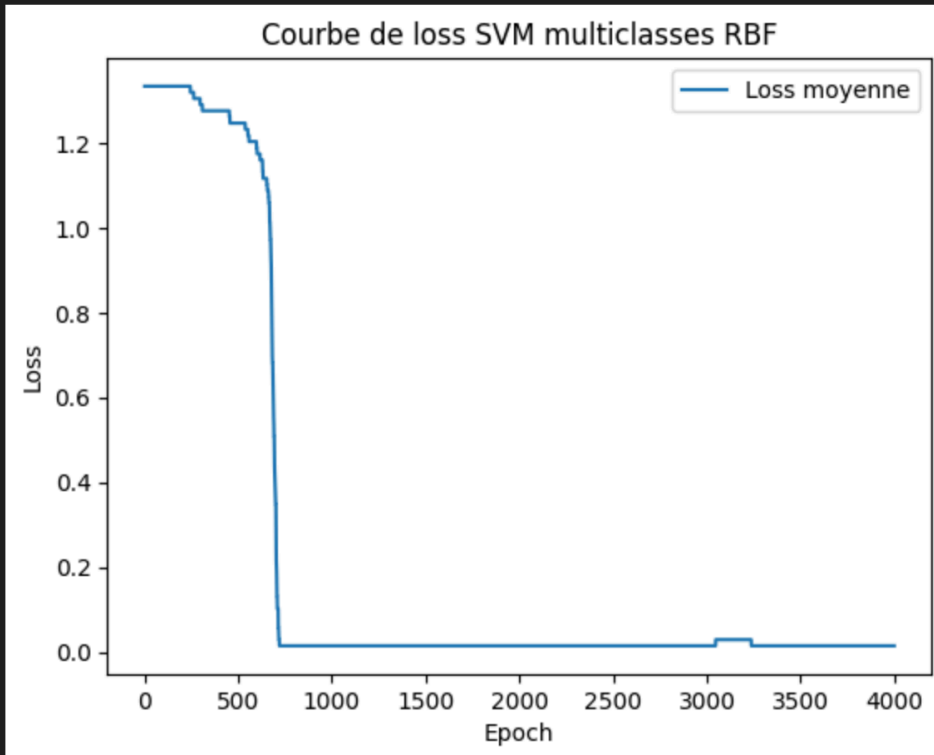
Qu'est-ce qui pourrait poser ce type de problème?

Nous avons plusieurs piste à observer :

- Sur-apprentissage : Le modèle n'a peut-être pas appris à différencier les autres classes.
- Sous-apprentissage : Le modèle est peut être trop simple ou n'a pas bien été entraîné.
- Problème de Preprocessing : Donnée mal normalisée, mal dimensionnée.

D'après les tests faits notamment ceux donnés en classe, le modèle fonctionne pour des problème linéaire, on a aussi vu qu'il arrivait à résoudre les problèmes non-linéaire comme par exemple le XOR avec noyau RBF, et on a vu qu'il gérait très bien le multiclasse avec des résultats a 98%. Donc on peut s'orienter vers le problème de preprocessing.

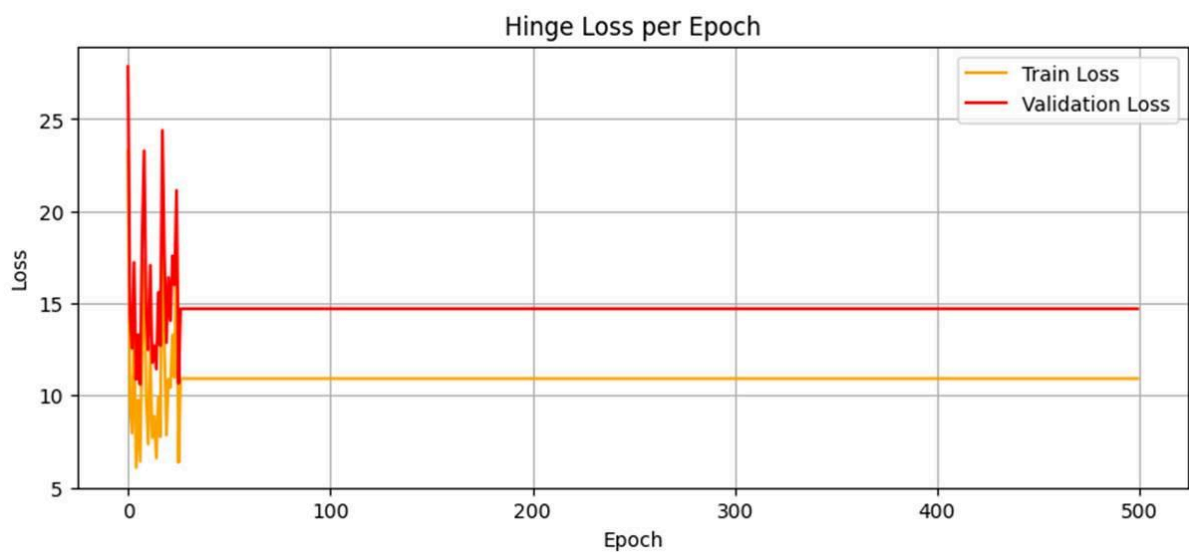
Ici on observe un overfitting qui est probablement du a une mauvaise normalisation, cette étape est donc a retaravailler.

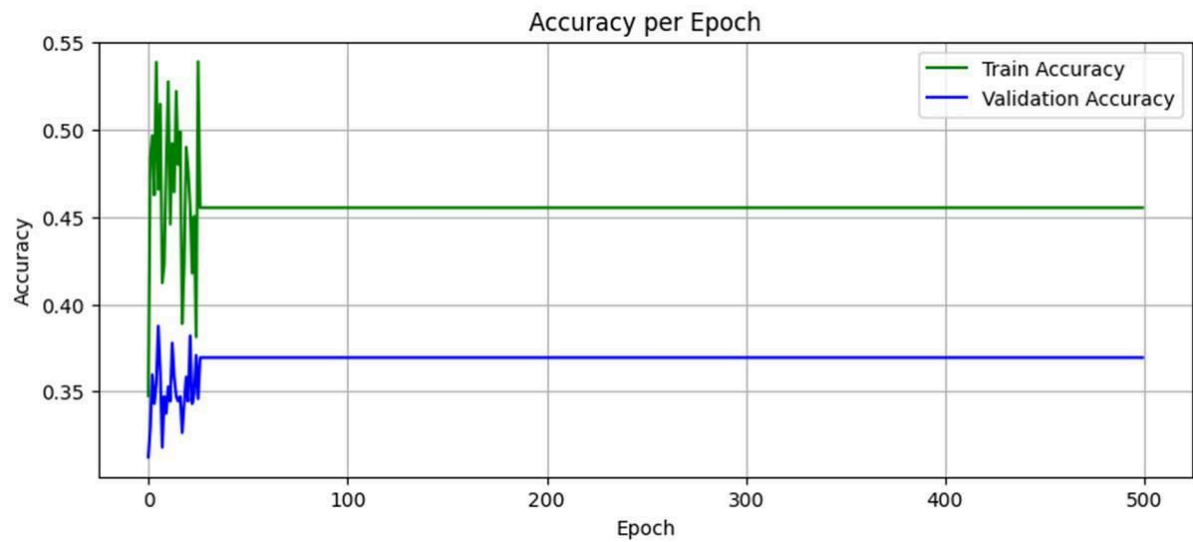


Ici nous avons comme valeurs pour paramètre :

- Epochs = 500
- Gamma = 1.0
- Alpha = 10^{-5}

Et nous avons ces courbes :

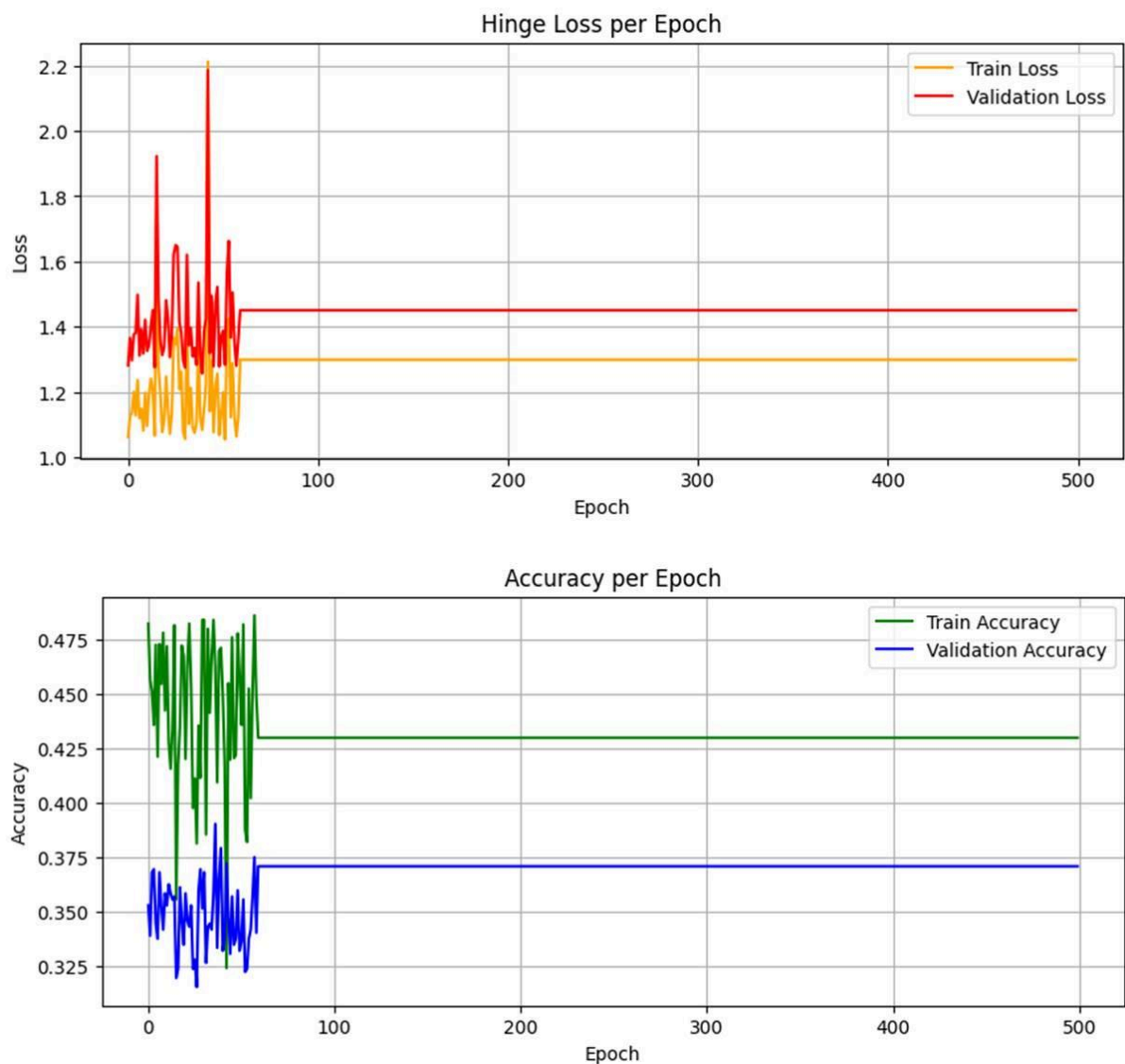




Ici nous avons comme valeurs pour paramètre :

- Epochs = 500
- Gamma = 0.5
- Alpha = 10^{-3}

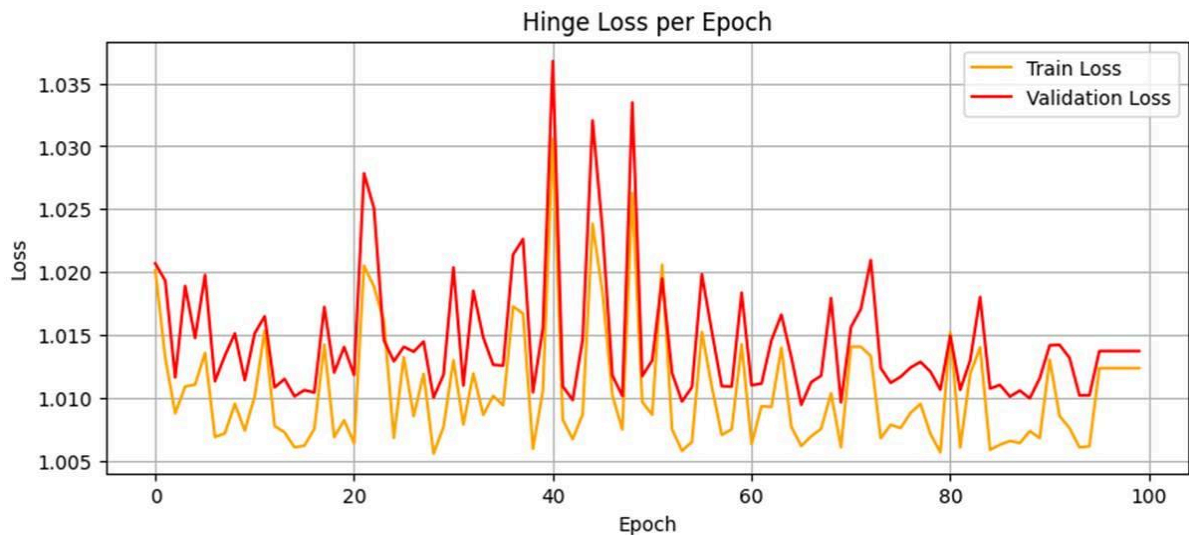
Et nous avons ces courbes :



Ici on peut voir que ces changements ont fait en sorte que le modèle soit plus régulier, moins surentraîné et plus performant mais cela n'est pas encore satisfaisant mais cela nous permet d'orienter notre développement.

Maintenant pour ces paramètre :

- epochs = 100
- gamma = 0.1
- learning_rate = 0.01
- alpha = 1e-2

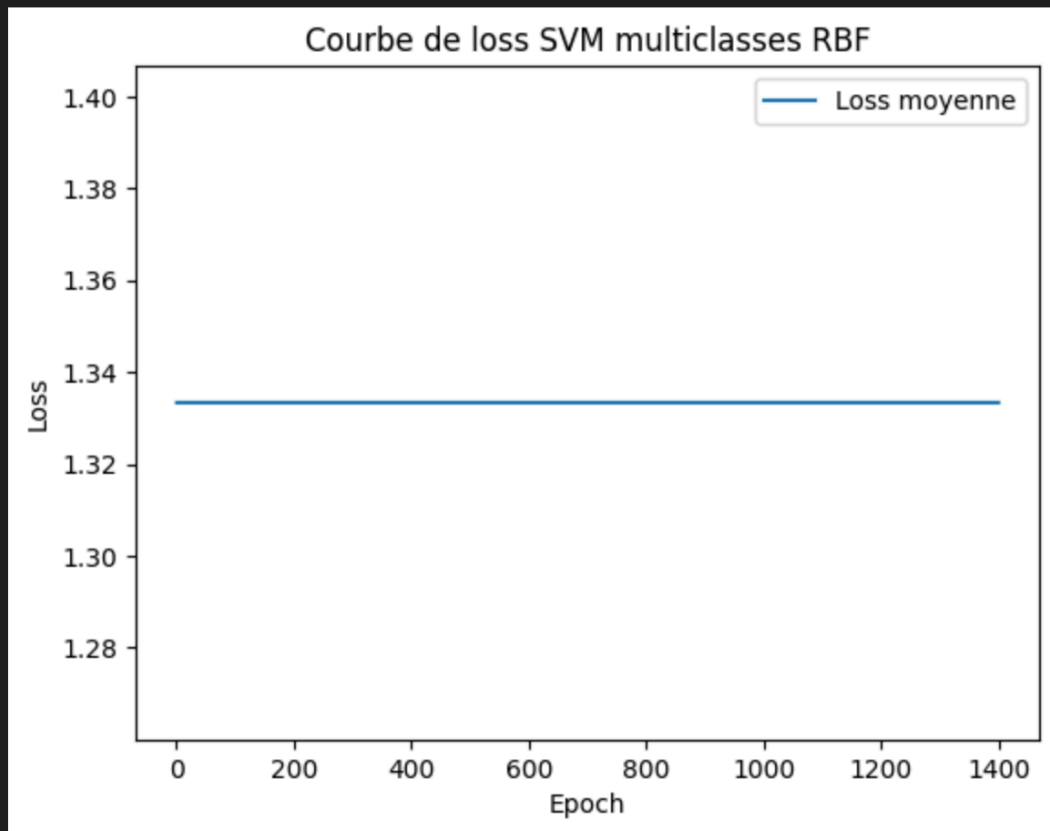


Analyse du comportement du modèle :

On observe que le modèle apprend, mais pas de manière parfaitement régulière. Du côté de la *validation loss*, les fluctuations sont nettement plus marquées. Cela suggère que le modèle n'est plus en situation de surapprentissage, mais que la validation reste instable. Cette instabilité pourrait être due à un dataset trop petit ou trop bruité, ou encore à certains hyperparamètres notamment γ ou $n_components$ qui seraient fixés à des valeurs trop faibles.

Contexte technique :

Cette phase s'est révélée particulièrement chronophage, en grande partie à cause d'un Huawei Mate X Pro 2021 capricieux ces dernières semaines. L'appareil a présenté des écrans bleus fréquents et des extinctions intempestives, ce qui a fortement perturbé le bon déroulement des expérimentations.



Accuracy TRAIN : 100.00%

Accuracy TEST : 33.33%

Nous avons un exemple d'un model qui n'apprends rien avec une courbe qui reste parfaitement plate durant tous les epochs

MLP(Multi-Layer Perceptron) :

Plusieurs modèles de type **MLP (Multi-Layer Perceptron)** ont été implémentés en Rust. Ces modèles sont conçus pour traiter des données d'images converties en vecteurs de pixels, et permettent d'effectuer des tâches de classification (binaire ou multiclasse) ainsi que de régression.

Trois variantes principales sont développées :

- Un **MLP simple** pour la régression ou la classification binaire.
- Un **MLP multiclasse (MLPClassifier)** avec softmax.
- Un **MLP profond (MLPDeepClassifier)** avec plusieurs couches cachées et choix de l'activation (ReLU ou Tanh)

1. MLP simple (binaire ou régression)

La première version de notre modèle MLP (Multi-Layer Perceptron) est conçue pour faire des choses simples, comme **reconnaître deux classes** (par exemple la lettre A ou B), ou faire de la **régression** (prédire une valeur continue, comme un score ou une position).

C'est une version très basique, avec une seule **couche cachée** et **un seul neurone en sortie**. Cette structure permet de commencer facilement, sans complexité inutile.

Ce modèle peut fonctionner de deux façons :

- soit avec **une fonction d'activation tanh**, pour donner plus de souplesse à l'apprentissage,
- soit sans activation du tout (comportement linéaire), ce qui est plus simple mais moins puissant.

Quand on donne une image ou un vecteur en entrée, le modèle fait une **propagation avant** : chaque valeur est multipliée par un poids, puis on ajoute un biais. Le résultat passe éventuellement dans une activation (comme **tanh**), et la **sortie finale est un nombre** (par exemple 0.8, si le modèle pense à 80% que c'est la classe A).

La propagation avant se fait selon :

$$z = W \cdot x + b$$

$$a = \tanh(z)$$

$$\hat{y} = w_{\text{out}} \cdot a + b_{\text{out}}$$

Fonction de perte (MSE) :

$$\text{MSE} = (1/N) * \sum (\hat{y}_i - y_i)^2$$

Dérivée de tanh :

$$\tanh'(x) = 1 - \tanh^2(x)$$

Mise à jour des poids :

$$w \leftarrow w - \eta \times \nabla \text{MSE}$$

Pendant l'entraînement, la fonction **fit** ajuste les poids du réseau pour qu'il fasse moins d'erreurs. Le modèle utilise l'**erreur quadratique moyenne (MSE)** pour mesurer la différence entre ce qu'il prédit et ce qu'on attend. Plus la MSE est petite, mieux c'est.

Ensuite, on utilise la **rétropropagation** pour corriger les erreurs :

le modèle regarde combien il s'est trompé, puis **remonte dans le réseau** pour ajuster les poids petit à petit. Il utilise pour cela la dérivée de la fonction d'activation (par exemple **$1 - \tanh^2(x)$** pour tanh).

Chaque poids est mis à jour en suivant la règle : **$w \leftarrow w - \eta \times \text{erreur}$** ,

où **η** est le **taux d'apprentissage**, c'est-à-dire à quel point on change les poids à chaque correction.

Enfin, une fois le modèle entraîné, on peut utiliser la fonction **predict** pour lui donner une nouvelle entrée et obtenir une prédiction.

Ce petit modèle est une **très bonne base pour comprendre les bases des réseaux de neurones** : comment les données circulent (propagation), comment on apprend à corriger les erreurs (rétropropagation), et comment les poids sont ajustés. Il prépare le terrain pour passer ensuite à des modèles plus complexes.

2. MLP multiclasse avec Softmax (MLPClassifier)

La deuxième version du réseau de neurones utilisée dans ce projet est conçue pour **reconnaître plusieurs classes à la fois** (par exemple les lettres A, B et C). Elle s'appelle **MLPClassifier** et fonctionne comme une version plus évoluée du MLP simple. Cette fois, le réseau a **plusieurs couches cachées**, ce qui lui permet de mieux comprendre les relations complexes entre les caractéristiques des images en entrée.

En sortie, le réseau possède **un neurone par classe**. Chaque neurone donne un score indiquant à quel point l'entrée correspond à cette classe. Pour rendre ces scores compréhensibles, le modèle utilise une **fonction appelée softmax**, qui transforme tous les scores en **probabilités** (des valeurs entre 0 et 1 qui s'additionnent à 1). Grâce à cela, le modèle peut dire : "je pense que c'est 80% la lettre B, 15% la A et 5% la C", par exemple.

Pendant l'apprentissage, la fonction **fit** entraîne le réseau avec les données. Pour chaque exemple, il :

- fait une **prédiction** avec propagation avant
- **compare** la prédiction à la vraie réponse (représentée avec un vecteur "one-hot", c'est-à-dire [1, 0, 0] si c'est la classe A)
- puis il **corrige ses erreurs** avec la rétropropagation et **met à jour ses poids** pour s'améliorer petit à petit

Même si le calcul des erreurs se fait en interne avec le softmax, le modèle utilise aussi une **MSE (erreur quadratique moyenne)** pour vérifier qu'il progresse bien à chaque époque

Le modèle utilise la **fonction tanh** comme activation dans les couches cachées, ce qui aide à rendre les calculs plus flexibles. À chaque étape, les poids sont modifiés selon une règle simple :

$\mathbf{w} \leftarrow \mathbf{w} - \eta \times \text{erreur}$, où η est le **taux d'apprentissage**, c'est-à-dire à quel point on change les poids à chaque erreur.

Enfin, à chaque époque, le modèle enregistre les **performances** : précision sur les données d'entraînement et de test, et courbe d'erreur (MSE). Cela permet de voir si le réseau apprend bien ou s'il commence à trop coller aux données (surapprentissage)

Activation softmax en sortie :

$$\hat{y}_i = e^{z_i} / \sum e^{z_j} \quad \text{pour } j = 1 \text{ à } C$$

Fonction de perte (toujours MSE) :

$$\text{MSE} = (1/N) * \sum ||\hat{y}_i - y_i||^2$$

Mise à jour des poids :

$$w \leftarrow w - \eta \times \nabla \text{MSE}$$

3. MLP profond (MLPDeepClassifier)

Le MLP profond est la version la plus avancée du réseau de neurones utilisée pour le MLP. Il est conçu pour mieux apprendre des données complexes en utilisant plusieurs couches cachées, ce qui le rend plus puissant. On peut choisir le nombre de couches, le nombre de neurones par couche et même la fonction d'activation utilisée (parmi Relu ou Tanh)

Les sorties du modèle sont comparées à des cibles codées en "one-hot", c'est à dire des vecteurs comme [1,-1,-1] si la première classe est la bonne

L'apprentissage se fait en **mini-batches**, ce qui veut dire que le modèle apprend petit à petit, avec des groupes d'exemples au lieu de tout traiter d'un coup. Cela aide à rendre l'entraînement plus stable. De plus, une **régularisation L2** est ajoutée : c'est une technique qui empêche le modèle de "trop apprendre" les données (surapprentissage), en évitant que les poids deviennent trop grands.

À chaque passage, le modèle fait une **propagation avant** pour faire une prédiction, puis une **rétropropagation** pour corriger ses erreurs. Les poids sont mis à jour en fonction des erreurs, mais aussi en prenant en compte la régularisation.

Pour suivre l'évolution de l'apprentissage, le modèle affiche régulièrement les scores de précision (accuracy) et la perte (loss) sur les données d'entraînement et de test. Toutes les 10 époques, un message est affiché pour montrer si le modèle progresse bien. Ce MLP profond est donc une solution plus complète et plus performante, bien adaptée à des tâches de classification multiclasse comme reconnaître plusieurs lettres de l'alphabet en langue des signes

Propagation dans une couche cachée :

$$a^{(l)} = \sigma(W^{(l)} \cdot a^{(l-1)} + b^{(l)})$$

où σ est soit ReLU, soit tanh

Fonction de perte avec régularisation L2 :

$$\text{Loss} = \text{MSE} + \lambda \times \sum ||W^{(l)}||^2$$

Mise à jour des poids avec régularisation :

$$W \leftarrow W - \eta \times (\nabla \text{Loss} + \lambda \times W)$$

Résultats expérimentaux sur le dataset:

Résultats obtenus après entraînement :

Problème 1 – Prédictions toujours identiques

Au tout début de nos expérimentations, le modèle MLP présentait un comportement anormal : il prédisait toujours la même lettre, quelle que soit l'image en entrée (par exemple, toujours "A"). Ce type de sortie constante est le signe que le modèle n'arrive pas à différencier les classes, et qu'il ne fait en réalité aucun apprentissage significatif.

En parallèle, les logs d'entraînement montraient une stagnation rapide de la fonction de perte (loss), qui se fixait autour de 0.0125 dès les premières centaines d'époques. Les poids du réseau restaient quasiment constants, ce qui prouvait que les gradients étaient trop faibles ou que les données n'apportent pas assez de diversité ou de signal utile à l'apprentissage.

Ces deux symptômes – sorties identiques et loss bloquées – sont étroitement liés. Ils reflètent un apprentissage bloqué dès le départ, souvent causé par :

- un manque de normalisation des images,
- une architecture trop simple (pas assez de neurones ou de couches),
- ou un jeu de données trop peu varié ou déséquilibré.

Ce constat nous a permis d'identifier des axes clairs d'amélioration, appliqués dans la suite du projet

Solution 1 :

Pour corriger cela, nous avons apporté trois améliorations majeures :

Normalisation des images : toutes les images ont été mises à la même échelle (valeurs entre 0 et 1), ce qui facilite l'apprentissage et évite que certaines dimensions dominent les autres.

Régularisation L2 : nous avons ajouté une pénalité sur la taille des poids du réseau, ce qui empêche le modèle de trop s'adapter aux données d'entraînement et l'aide à mieux généraliser.

Augmentation des données (via variété d'images) : nous avons enrichi les données en prenant des images dans différents environnements (conditions de lumière, angles différents), ce qui améliore la robustesse du modèle.

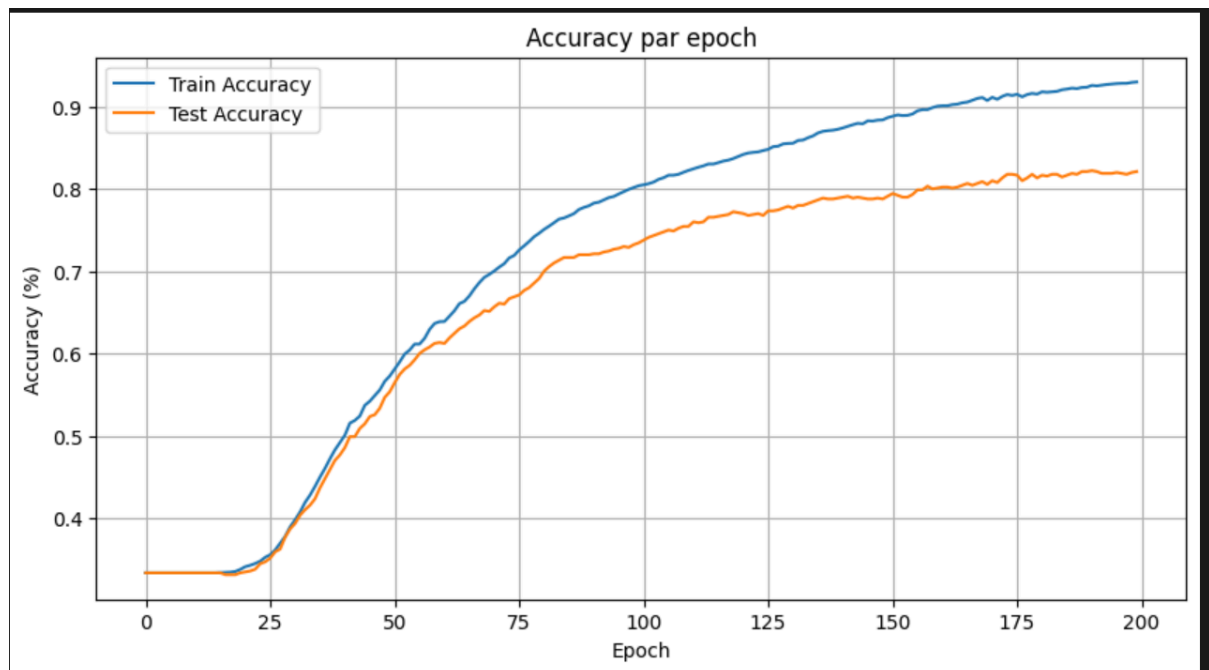
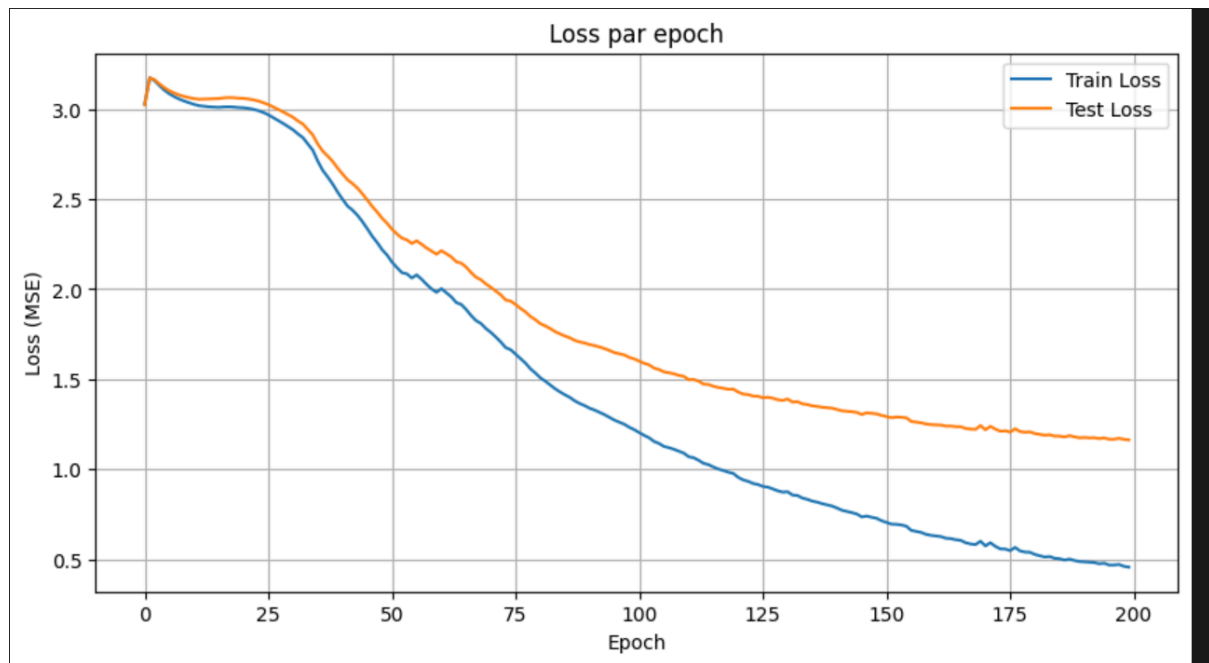
Problème 2 – Précision faible malgré un entraînement approfondi:

Même après avoir stabilisé l'apprentissage du modèle MLP profond, nous avons observé une **précision relativement basse de 38% sur les données de test**. Le modèle prédisait mieux qu'avant, mais restait loin d'une reconnaissance fiable. Pourtant, nous avons utilisé des paramètres classiques mais raisonnablement puissants :

- **2 couches cachées de 64 neurones chacune,**
- **taux d'apprentissage (learning rate) de 0.01,**
- **régularisation L2 de 0.001,**
- **batch_size de 32,**
- **l'activation ReLU**

L'entraînement a été conduit sur **300 époques**, avec un suivi précis de la perte et de la précision à chaque étape.

Les courbes montrent bien une **diminution continue de la loss** et une **progression de l'accuracy** sur le train et le test, mais cette montée s'est stabilisée autour de 83% (train) et **80% (test)**, avec une **précision finale mesurée à seulement 38%** lors de la prédiction réelle sur le jeu de test. Cela révèle un **écart entre les métriques globales et la performance réelle**, souvent causé par des classes mal apprises, une généralisation imparfaite, ou encore un modèle qui n'arrive pas à bien exploiter la structure des images



Solution 2 :

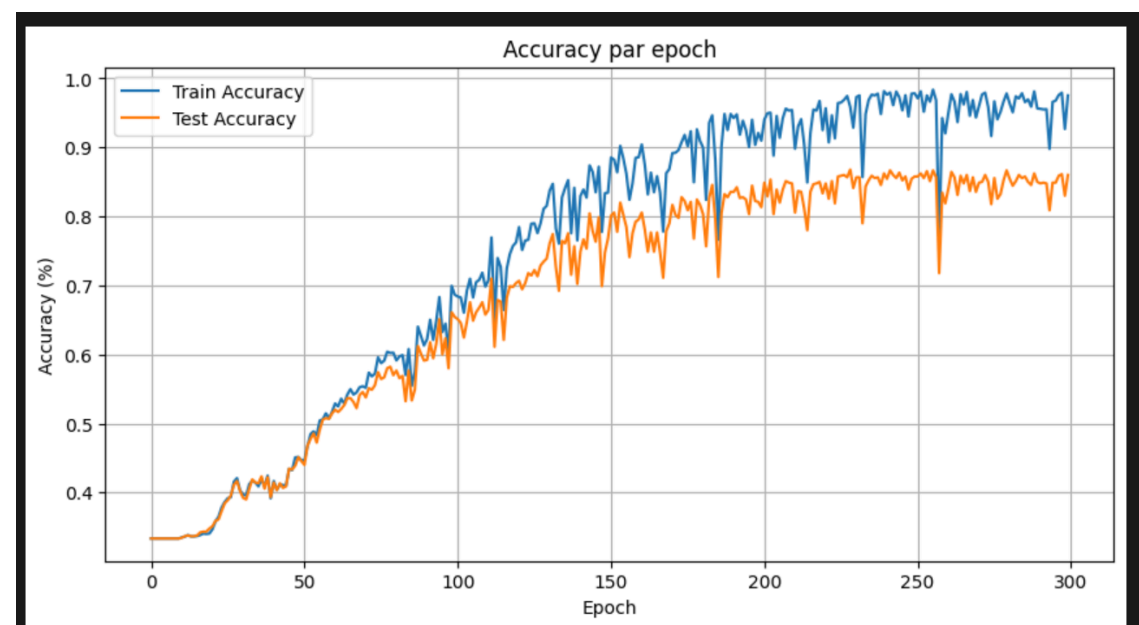
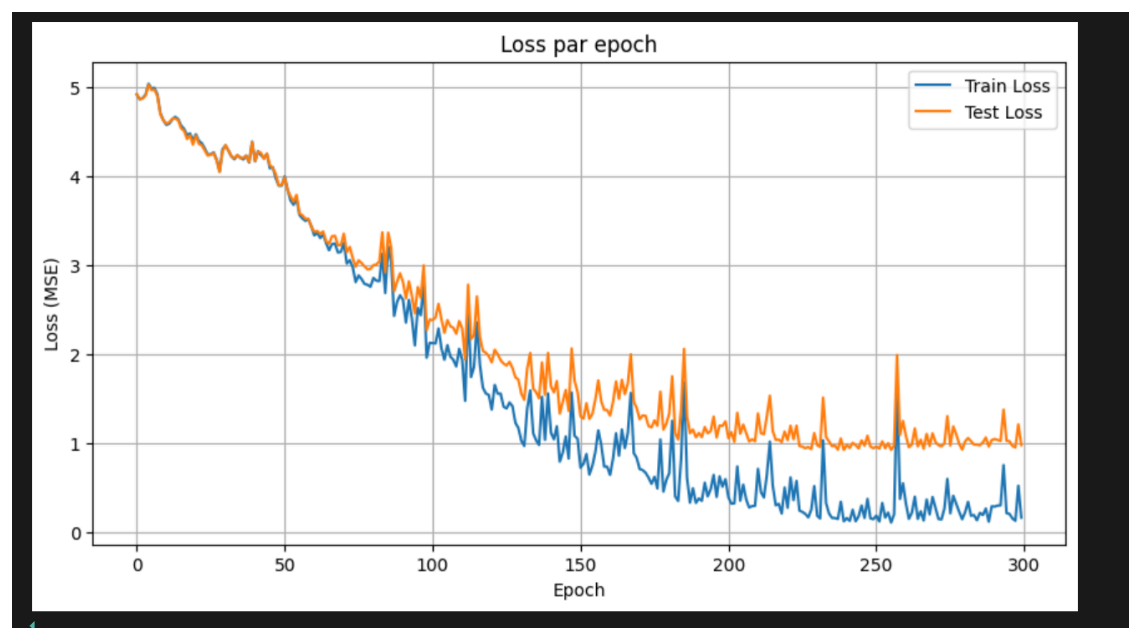
Nous avons ajusté plusieurs paramètres du MLP profond en utilisant la fonction d'activation Relu :

- Taux d'apprentissage (**learning_rate**) augmenté à **0.01**
- Nombre d'époques (**epochs**) augmenté à **300**

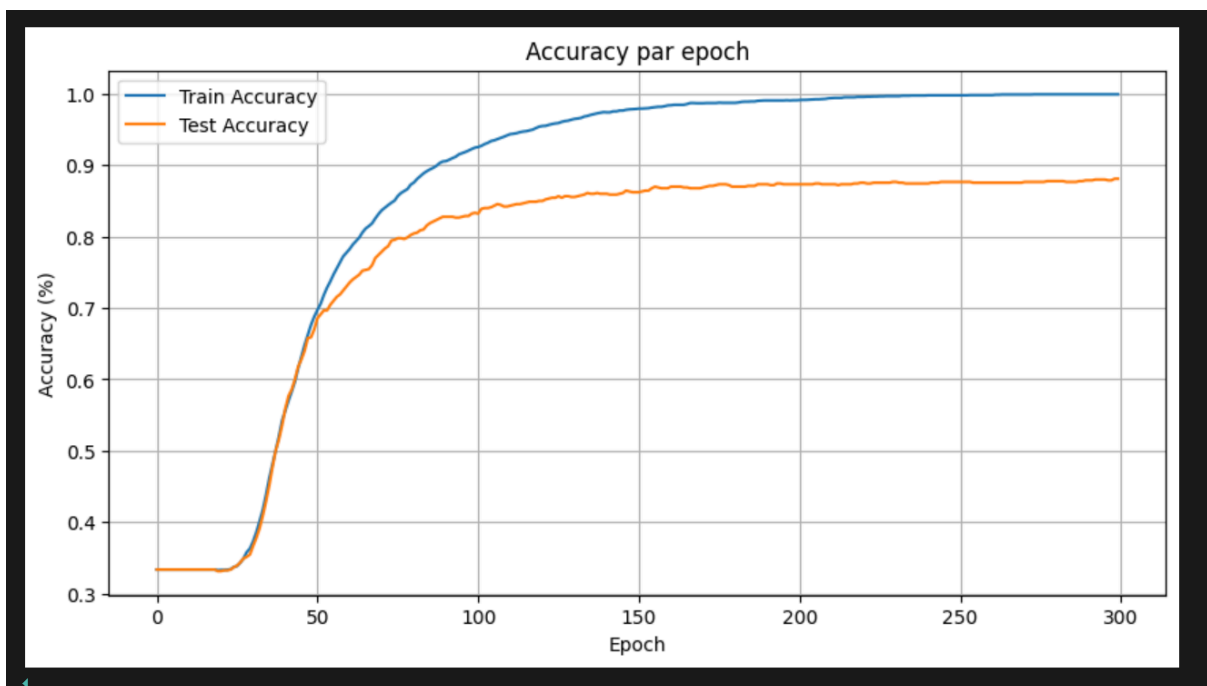
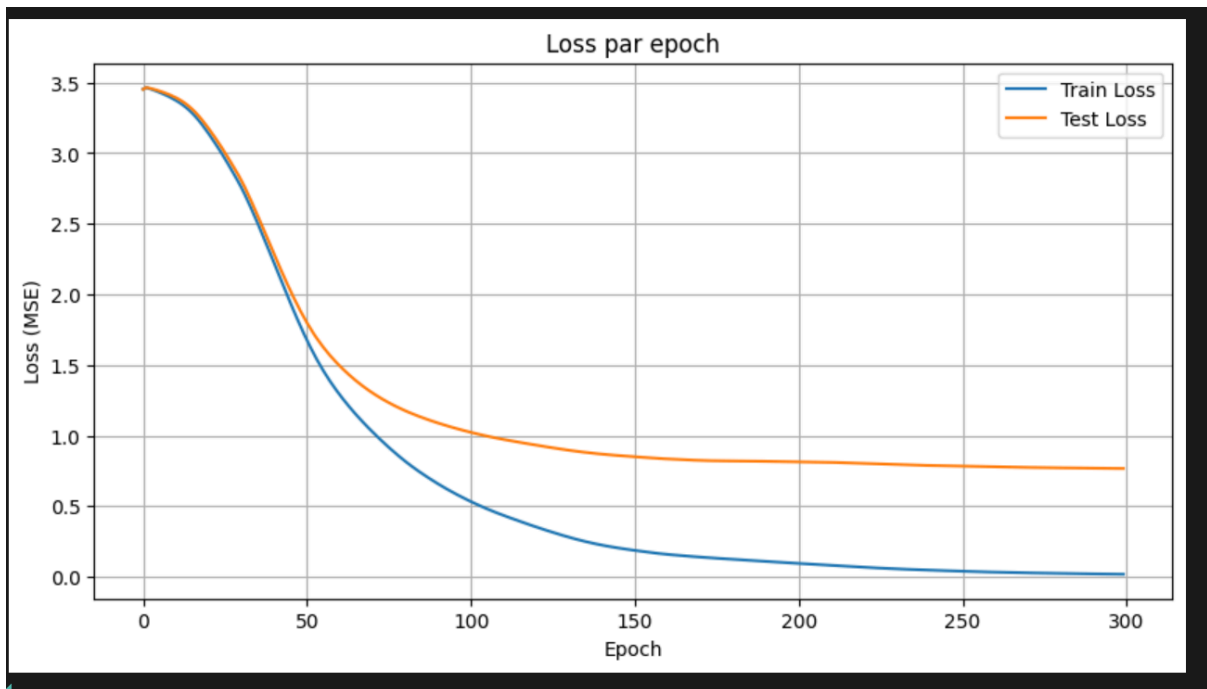
Ces changements permettent au modèle :

- d'apprendre plus efficacement grâce à un meilleur équilibre entre vitesse et stabilité,
- d'exploiter plus d'exemples pendant chaque mise à jour (mini-batches)

Grâce à ces modifications, la précision sur les données de test est passée de **38%** à **environ 62%**, ce qui montre une **forte amélioration des performances**. Le modèle a donc mieux appris à reconnaître les différentes lettres, tout en restant stable dans son apprentissage



Nous avons également testé avec la fonction d'activation Tanh qui a donné une précision de 66% :



En affichant les images de tests :



Modèle	MLP Deep	MLP Deep	MLP Deep
Tache	Multiclasse	Multiclasse	Multiclasse
Activation	Tanh	Relu	Relu
Nb hidden layers	2	2	2
Nb hidden units	64	64	64
Nb Epochs	300	200	300
Learning rate	0.001	0.001	0.01
Accuracy	66%	38%	62%

Réseau à Fonctions de Base Radiale (RBFN) :

Le modèle RBFN est une architecture de type "réseau à une couche cachée" dont les neurones sont basés sur une fonction de similarité radiale, typiquement une gaussienne. Chaque neurone caché mesure la distance entre une entrée et un centre, ce qui permet une modélisation non-linéaire des données

L'implémentation permet de traiter trois cas d'usage bien distincts : la régression, la classification binaire, et la classification multiclasse

Régression et Classification Binaire :

Dans les cas de régression et de classification binaire, l'apprentissage s'effectue via une solution analytique. Les centres sont initialisés directement à partir des données d'entrée (chaque exemple devient un centre), et la matrice de transformation Φ (phi), calculée à partir des distances gaussiennes entre les entrées et les centres, est utilisée pour résoudre l'équation normale. Plus précisément :

$$W = (\Phi^T \Phi)^{-1} \Phi^T y$$

W: sont les poids de sortie

y : les sorties attendues

Cette solution est rapide, élégante et ne nécessite pas d'itérations. Une fois les poids calculés, la prédiction s'obtient par un simple produit scalaire entre l'entrée transformée et les poids. La classification binaire utilise ensuite un seuil (positif/négatif) pour attribuer une étiquette

Classification Multiclasse (Descente de Gradient)

Pour les problèmes multiclasse, l'apprentissage utilise une descente de gradient. Les sorties sont représentées en one-hot encoding, et les poids sont mis à jour sur plusieurs époques selon le gradient de l'erreur :

$$W \leftarrow W + (\eta / N) \Phi^T (Y - \hat{Y})$$

W : sont les poids de sortie

y : les sorties attendues.

Cette solution est rapide, élégante et ne nécessite pas d'itérations. Une fois les poids calculés, la prédiction s'obtient par un simple produit scalaire entre l'entrée transformée et les poids. La classification binaire utilise ensuite un seuil (positif/négatif) pour attribuer une étiquette

Résultats obtenu :

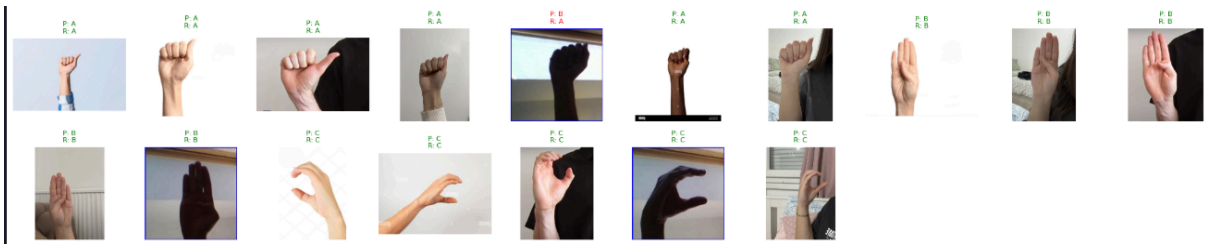


Tableau d'expérimentations :

<u>Learning Rate</u>	0.1	0.1	0.01
Epochs	100	100	100
<u>sigma</u>	1.0	3.0	1.0
<u>Accuracy</u>	0.8923076923076924	0.50216780319790	0.6221335012540

Interface Web Interactive pour la Démonstration des Modèles de Machine Learning

L'interface web développée permet de sélectionner un modèle parmi : Linear, MLP, MLP Deep, RBF et SVM. Une fois le modèle choisi, on peut configurer les hyperparamètres comme le learning rate, le nombre d'epochs, ou le sigma pour le RBF. Après entraînement, l'utilisateur peut charger une image pour obtenir une prédiction en temps réel

Les résultats d'entraînement sont affichés sous forme de courbes de loss par epoch. Chaque exécution (run) est identifiée par un nom unique et sa courbe peut être comparée aux autres. Cela permet d'observer l'évolution de la perte, d'évaluer la stabilité et l'efficacité du modèle entraîné.

L'interface inclut aussi des boutons pour sauvegarder et recharger des modèles, facilitant la réutilisation sans devoir réentraîner à chaque fois

Le développement de cette application a nécessité plusieurs ajustements techniques, notamment à cause des contraintes liées aux bindings entre Rust, Python (via FFI) et FastAPI. L'un des problèmes récurrents a été les erreurs de typage lors de la transmission de tableaux ou de pointeurs, notamment pour les couches cachées dans le MLP Deep. Des erreurs comme `TypeError: Don't know how to convert parameter` ont souvent nécessité des modifications dans la définition des fonctions C exposées via `#[no_mangle]`, ainsi que dans les signatures `ctypes` côté Python.

D'autres difficultés sont apparues lors de l'ajout de nouvelles fonctionnalités à l'API. Par exemple, l'introduction du suivi de la loss et de l'accuracy pour les modèles profonds a entraîné des erreurs inattendues (access violation, crash silencieux ou mauvais pointeurs retournés). À chaque amélioration, il fallait s'assurer que la compatibilité entre les structures de données Rust et leur représentation côté Python était bien respectée.

Enfin, l'intégration avec l'interface React n'a pas été immédiate. Certaines erreurs venaient du mauvais typage des données envoyées à l'API ou du décalage entre le format des résultats (JSON, tableaux) et les besoins des composants graphiques. Cela a nécessité plusieurs itérations de test et de refactorisation pour stabiliser l'ensemble

Modele

MLP

Linear

MLP

MLP Deep

RBF

SVM

Epochs

10

START TRAINING

Model Save / Load

Model name

SAVE MODEL

Available Models

Select image for prediction

UPLOAD AN IMAGE

Training results

No data to display

Modele

RBF

Learning Rate

0,01

Sigma

1

Epochs

10

START TRAINING

Model Save / Load

Model name

SAVE MODEL

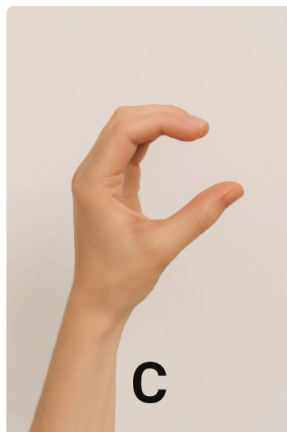
Available Models

LOAD MLP.MODEL

LOAD MLP2.MODEL

Select image for prediction

UPLOAD AN IMAGE



PREDICT

Prediction : C



80 %

 PRÉDIRE

 Prédiction : B

Modèle

P

Couches

Arbres couche 1

+ AJOUTER UNE COUCHE

Learning Rate

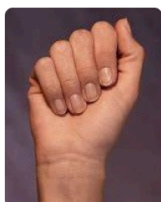
0.1

Époques

LANCER L'ENTRAÎNEMENT

📁 Sélection d'image pour prédiction

 IMPORTER UNE IMAGE



PRÉDIRE

 Prédiction : A

Résultats d'entraînement

RUN_MLP_1752777190227

run_mlp_1752777190227

Résultats d'entraînement

RUN_MLP_1752918946779

RUN_MLP_1752919186562

RUN_MLP_1752919199688

